

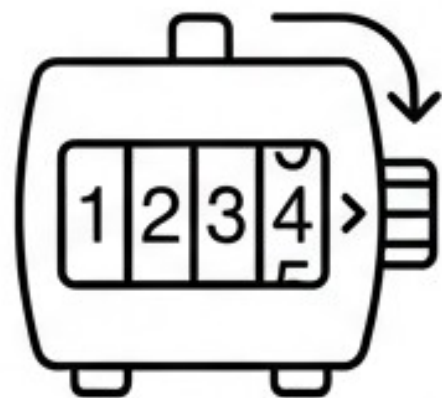
Приложение на циклични конструкции. Цикъл for



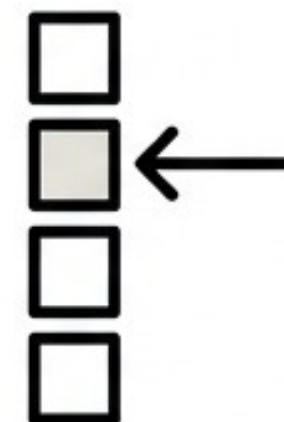
```
for student in class:  
    print('Успех!')
```

Защо ни е необходим цикъл **for**?

За разлика от командата `while`, цикълът `for` се използва най-вече в две конкретни ситуации:



Когато искаме повторенията да се извършат **определен брой пъти**.



Когато искаме да вземем последователно елементите от **даден списък**.

Анатомия и логика на цикъла

```
for име_на_променлива in списък :  
    команда 1  
    команда 2
```

Правило: Целта е блокът от команди (с отместване навътре) да се изпълни толкова пъти, колкото елемента има в списък.



Обхождане на списък: Примерът с океаните

Code Terminal	Output
<pre>oceans = ['Атлантически', 'Тихи', 'Индийски', 'Южен', 'Северен ледовит'] for x in oceans: print(x)</pre>	<pre>Атлантически Тихи Индийски Южен Северен ледовит</pre>

По време на всяко повторение програмата взема поредния елемент от списъка и го запазва в променливата **x**, след което изпълнява командата **print(x)**.

Генераторът на числа: Функция range()

```
for x in range(10):  
    print(x)
```

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Включени (10 на брой)



Внимание: Функцията връща поредица от числа, започваща от 0 (по подразбиране). Последното число (стоп стойността) **НЕ се включва** в поредицата!

Контрол над range(): Начало и Край

```
for x in range(2, 8):  
    print(x)
```

2 3 4 5 6 7



Можем да променим началната стойност, ако не искаме тя да е 0, като подадем два параметъра: range(Начало, Край). Числото 8 отново не е включено.

Стъпка на нарастване в range()

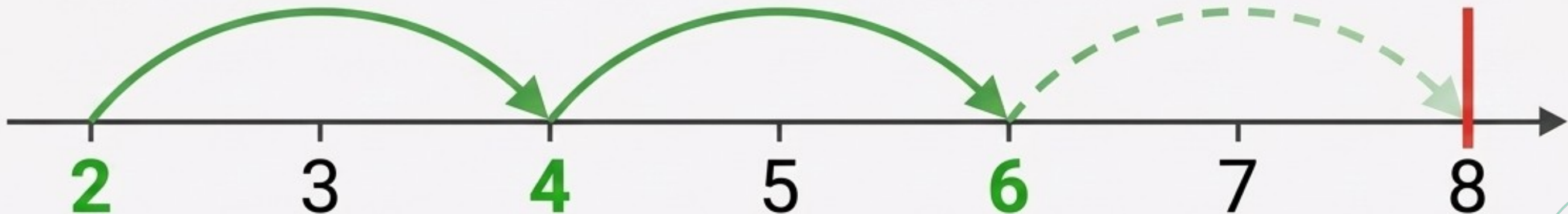
The Interactive Textbook

range(Начало, Край, Стъпка)

range(2, 8, 2)

```
for i in range(2, 8, 2):  
    print(i)
```

2 4 6



Третият параметър определя през колко числа да прескачаме. Съгласно параметрите ще отпечатаме числата от 2 до 8 със стъпка 2.

Тайният символ: Долна черта (_)



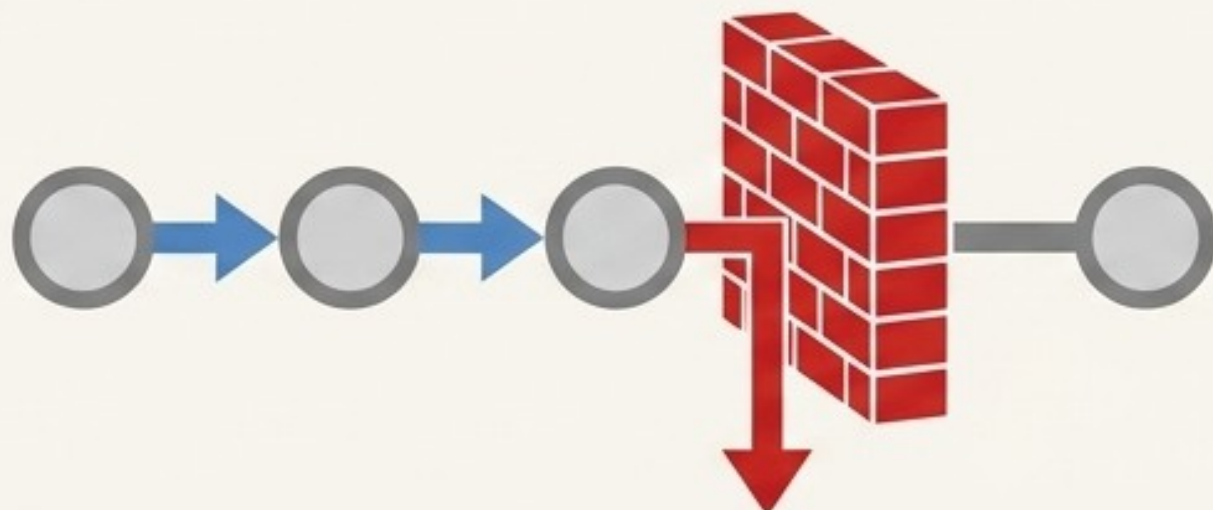
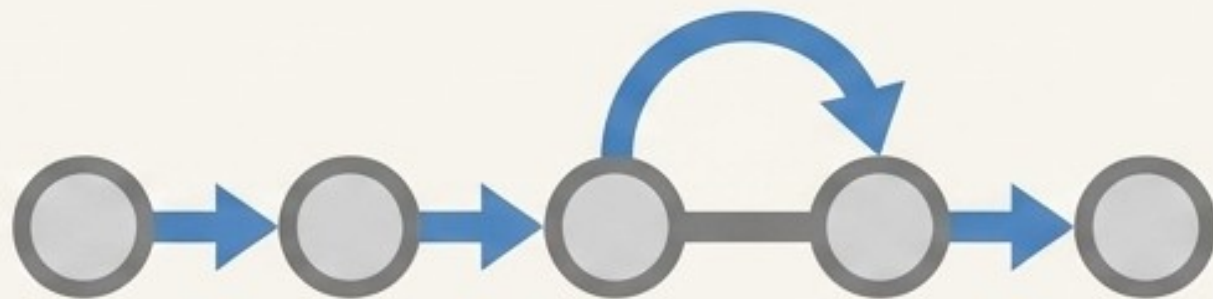
Pro Tip

```
for _ in range(5):  
    print(5 * '#')
```

```
#####  
#####  
#####  
#####  
#####
```

Ако няма да използваме променливата-бройч вътре в самия цикъл, добра практика е да я заменим със символа долна черта (_). Така кодът е по-чист и четим.

Контрол на трафика: break и continue

	break		Прекъсва изпълнението на целия цикъл. Програмата излиза от него напълно.
	continue		Прекъсва само текущата стъпка и преминава директно към следващото завъртане.

Важно: Тези команди действат по абсолютно същия начин, както и при цикъла while!

Практическа Мисия 1: Текстови фигури

Нарастваща фигура

Мисия: Отпечатване на триъгълник

```
for i in range(1, 8):  
    print(i * '*')
```

```
*  
**  
***  
****  
*****  
*****  
*****  
*****
```

Отпечатва триъгълник от звездички, като на всеки ред броят им расте.

Предизвикателство

Предизвикателство: Квадрат 10x10

```
for _ in range(10):  
    print(10 * '#')
```

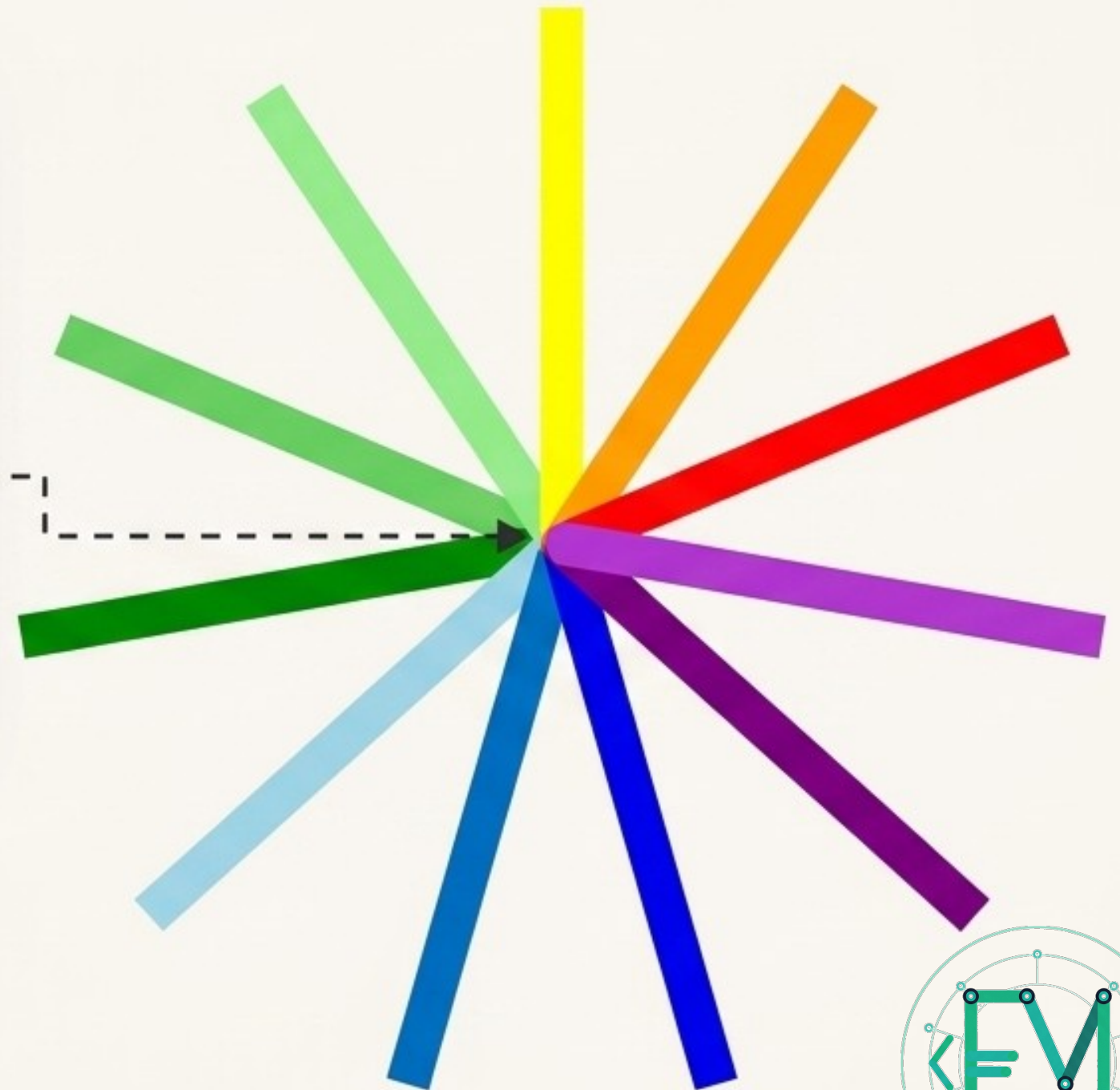
```
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####
```

Как да отпечатаме квадрат с размер 10x10, използвайки символа #?

Практическа Мисия 2: Цветна звезда (Turtle)

```
colors = ['yellow', 'orange', 'red',  
'violet', 'purple', 'blue', 'lightblue',  
'green', 'lightgreen']
```

```
for i in range(9):  
    # код за начертаване на лъч и смяна  
    на цвят
```



Цикълът **for** ни позволява да начертаем 9 лъча с различна дебелина и цвят, като обхождаме списъка с цветовете без да пишем едни и същи команди 9 пъти!

Обобщение (Cheat Sheet)



1. Предназначение.

Идеален за повторения точен брой пъти или за обхождане на елементи в списък.



2. Функция range().

range(стоп), range(старт, стоп), range(старт, стоп, стъпка)



3. Златното правило. При използване на range(), крайната стойност (стоп) НИКОГА не се включва в резултата!



4. Полезни трикове.

Използвайте _ вместо променлива, когато броячът не е нужен. Използвайте break за пълно спиране и continue за прескачане.